

✓ Checkpoint 3 — ASR: Data Preparation and Analysis

Datasets:

- `LibriSpeech test-clean` — English, read speech, ~2620 clips, ~5.4h [15]
- `FLEURS ru_ru test` — Russian, read speech [16]

Goals:

1. Load both datasets and inspect raw format
2. Preprocess to unified format (16kHz, mono, PCM-16)
3. EDA: duration, transcript, speech rate, character set, amplitude
4. Save manifests for Checkpoint 4

Dependencies:

```
pip install datasets==2.18.0 librosa==0.10.1 soundfile==0.12.1 pandas==2.2.1 matplotlib==3.8.3 tqdm==4.66.2
```

✓ 0. Setup

Target format : 16 KHZ, mono, PCM_16

```
!pip install datasets==3.6.0
```

```
Collecting datasets==3.6.0
  Downloading datasets-3.6.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.25.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.70.17)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.24.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (4.0.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.4.2)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.23.0)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.24.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (4.12.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2025.11.11)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2025.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (6.3.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.17.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (4.7.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.16.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.17.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (8.2.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (1.3.0)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (13.9.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (2.19.0)
Requirement already satisfied: mdurl>=0.1 in /usr/local/lib/python3.12/dist-packages (from datasets==3.6.0) (0.1.2)
Downloading datasets-3.6.0-py3-none-any.whl (491 kB)
491.5/491.5 kB 5.6 MB/s eta 0:00:00

Installing collected packages: datasets
  Attempting uninstall: datasets
    Found existing installation: datasets 4.0.0
    Uninstalling datasets-4.0.0:
      Successfully uninstalled datasets-4.0.0
  Successfully installed datasets-3.6.0
```

```
import os
import random
import unicodedata
from pathlib import Path
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import soundfile as sf
import librosa
from datasets import load_dataset
from tqdm.auto import tqdm
```

```
SEED = 31122002
random.seed(SEED)
np.random.seed(SEED)

PROCESSED_DIR = Path("data/processed")
MANIFEST_DIR = Path("data/manifests")
EDA_DIR = Path("eda/asr")
for d in [PROCESSED_DIR, MANIFEST_DIR, EDA_DIR]:
    d.mkdir(parents=True, exist_ok=True)

TARGET_SR = 16_000
TARGET_SUBTYPE = "PCM_16"
PEAK_TARGET = 0.9
```

1. Load Datasets

```
ls = load_dataset(
    "openslr/librispeech_asr",
    "clean",
    split="test",
    trust_remote_code=True,
)
print(len(ls))
print(ls.features)
```


Resolving data files: 100%	48/48 [00:00<00:00, 2554.23it/s]
Resolving data files: 100%	48/48 [00:00<00:00, 3344.80it/s]
clean/test/0000.parquet: 100%	350M/350M [00:02<00:00, 247MB/s]
clean/train.100/0000.parquet: 100%	470M/470M [00:03<00:00, 2.35GB/s]
clean/train.100/0001.parquet: 100%	481M/481M [00:05<00:00, 98.0MB/s]
clean/train.100/0002.parquet: 100%	463M/463M [00:08<00:00, 69.3MB/s]
clean/train.100/0003.parquet: 100%	464M/464M [00:09<00:00, 53.9MB/s]
clean/train.100/0004.parquet: 100%	487M/487M [00:04<00:00, 180MB/s]
clean/train.100/0005.parquet: 100%	453M/453M [00:06<00:00, 91.1MB/s]
clean/train.100/0006.parquet: 100%	461M/461M [00:07<00:00, 92.8MB/s]
clean/train.100/0007.parquet: 100%	452M/452M [00:07<00:00, 59.9MB/s]
clean/train.100/0008.parquet: 100%	465M/465M [00:10<00:00, 48.5MB/s]
clean/train.100/0009.parquet: 100%	445M/445M [00:04<00:00, 128MB/s]
clean/train.100/0010.parquet: 100%	454M/454M [00:10<00:00, 49.7MB/s]
clean/train.100/0011.parquet: 100%	432M/432M [00:03<00:00, 232MB/s]
clean/train.100/0012.parquet: 100%	457M/457M [00:11<00:00, 56.3MB/s]
clean/train.100/0013.parquet: 100%	450M/450M [00:11<00:00, 39.5MB/s]
Downloading data: 100%	48/48 [04:29<00:00, 5.27s/files]
clean/train.360/0000.parquet: 100%	475M/475M [00:03<00:00, 172MB/s]
clean/train.360/0001.parquet: 100%	487M/487M [00:03<00:00, 157MB/s]
clean/train.360/0002.parquet: 100%	509M/509M [00:03<00:00, 285MB/s]
clean/train.360/0003.parquet: 100%	480M/480M [00:10<00:00, 52.3MB/s]
clean/train.360/0004.parquet: 100%	464M/464M [00:06<00:00, 98.6MB/s]
clean/train.360/0005.parquet: 100%	496M/496M [00:05<00:00, 132MB/s]

```
f1 = load_dataset(
    "google/fleurs",
    "ru_ru",
    split="test",
    trust_remote_code=True,
)
print(len(f1))
print(f1.features)
```

clean/train.360/0011.parquet: 100%	485M/485M [00:08<00:00, 51.7MB/s]
README.md: 13.3k/? [00:00<00:00, 728kB/s]	
clean/train.360/0012.parquet: 100%	456M/456M [00:10<00:00, 46.7MB/s]
fleurs.py: 12.5k/? [00:00<00:00, 800kB/s]	
clean/train.360/0013.parquet: 100%	497M/497M [00:13<00:00, 57.1MB/s]
data/ru_ru/audio/train.tar.gz: 100%	1.43G/1.43G [00:11<00:00, 158MB/s]
clean/train.360/0014.parquet: 100%	469M/469M [00:04<00:00, 216MB/s]
data/ru_ru/audio/dev.tar.gz: 100%	187M/187M [00:06<00:00, 71.4MB/s]
clean/train.360/0015.parquet: 100%	465M/465M [00:03<00:00, 492MB/s]
data/ru_ru/audio/test.tar.gz: 100%	433M/433M [00:09<00:00, 70.5MB/s]
clean/train.360/0016.parquet: 100%	481M/481M [00:04<00:00, 141MB/s]
train.tsv: 2.36M/? [00:00<00:00, 47.2MB/s]	
clean/train.360/0017.parquet: 100%	463M/463M [00:03<00:00, 151MB/s]
dev.tsv: 316k/? [00:00<00:00, 13.9MB/s]	
clean/train.360/0018.parquet: 100%	479M/479M [00:02<00:00, 286MB/s]
test.tsv: 735k/? [00:00<00:00, 25.4MB/s]	
clean/train.360/0019.parquet: 100%	456M/456M [00:06<00:00, 138MB/s]
Generating train split: 2562/0 [00:46<00:00, 120.69 examples/s]	
clean/train.360/0020.parquet: 100%	511M/511M [00:03<00:00, 271MB/s]
Generating validation split: 356/0 [00:05<00:00, 132.63 examples/s]	
clean/train.360/0021.parquet: 100%	491M/491M [00:05<00:00, 87.2MB/s]
Generating test split: 775/0 [00:11<00:00, 122.54 examples/s]	
clean/train.360/0022.parquet: 100%	497M/497M [00:03<00:00, 444MB/s]
clean/train.360/0023.parquet: 100%	487M/487M [00:04<00:00, 210MB/s]
clean/train.360/0024.parquet: 100%	468M/468M [00:02<00:00, 495MB/s]

```
s_ls = ls[10]
s_f1 = f1[10]

print("LibriSpeech:")
print(f"SR: {s_ls['audio']['sampling_rate']} Hz")
print(f"audio len: {len(s_ls['audio']['array'])} samples")

print("\nFLEURS:")
```

```
print(f"SR: {s_fl['audio']['sampling_rate']} Hz")
print(f"audio len: {len(s_fl['audio']['array'])} samples")
```

clean/train.360/0032.parquet: 100%	501M/501M [00:03<00:00, 188MB/s]
LibriSpeech:	
clean/train.360/0033.parquet: 100%	473M/473M [00:03<00:00, 241MB/s]
audio len: 48560 samples	
clean/train.360/0034.parquet: 100%	495M/495M [00:10<00:00, 110MB/s]
FLEURS:	
clean/train.360/0035.parquet: 100%	493M/493M [00:04<00:00, 504MB/s]
audio len: 241920 samples	
clean/train.360/0036.parquet: 100%	471M/471M [00:05<00:00, 83.3MB/s]

2. Preprocessing Pipeline

clean/train.360/0037.parquet: 100%	488M/488M [00:05<00:00, 258MB/s]
clean/train.360/0038.parquet: 100%	506M/506M [00:03<00:00, 192MB/s]

All three ASR candidates (Whisper Large-v3, Parakeet-TDT-0.6B-v3, Whisper Turbo) expect

- 16000 Hz sampling rate: 100%
 - mono channel: 100%
 - float32 input at inference time
- | | |
|------------------------------------|-----------------------------------|
| clean/train.360/0041.parquet: 100% | 490M/490M [00:03<00:00, 218MB/s] |
| clean/train.360/0042.parquet: 100% | 500M/500M [00:07<00:00, 94.6MB/s] |

We save as PCM-16 WAV on disk (lossless, half the size of float32).

clean/train.360/0043.parquet: 100%	492M/492M [00:08<00:00, 59.5MB/s]
------------------------------------	-----------------------------------

```
def preprocess(array: np.ndarray, src_sr: int) -> np.ndarray:
    """
    Resample to 16kHz, convert to mono, peak-normalize.
    """
    y = array.astype(np.float32)
    if src_sr != TARGET_SR:
        y = librosa.resample(y, orig_sr=src_sr, target_sr=TARGET_SR)
    if y.ndim > 1:
        y = librosa.to_mono(y)
    peak = np.max(np.abs(y))
    if peak > 0:
        y = y / peak * PEAK_TARGET
    return y
```

3. Build Manifests

For each clip we record:

- path to processed audio file on disk
- reference transcript
- language, duration, word count

This CSV is the direct input to the CP4 baseline notebook.

```
def build_manifest(
    dataset,
    transcript_key: str,
    language: str,
    prefix: str,
    out_dir: Path,
) -> pd.DataFrame:
    rows = []
    for i, sample in enumerate(tqdm(dataset, desc=f"{prefix}")):
        audio = sample["audio"]
        raw = np.array(audio["array"], dtype=np.float32)
        src_sr = audio["sampling_rate"]
        proc = preprocess(raw, src_sr)
        transcript = sample.get(transcript_key, "")
        fpath = out_dir / f"{prefix}_{i:05d}.wav"
        fpath.parent.mkdir(parents=True, exist_ok=True)
        sf.write(str(fpath), proc, TARGET_SR, subtype=TARGET_SUBTYPE)
        rows.append({
            "id": f"{prefix}_{i:05d}",
            "audio_path": str(fpath),
            "transcript": transcript,
            "language": language,
            "duration_sec": round(len(proc) / TARGET_SR, 4),
            "n_words": len(transcript.split()),
            "n_chars": len(transcript),
            "orig_sr": src_sr,
        })
    return pd.DataFrame(rows)
```

```
m_ls = build_manifest(ls, "text", "en", "ls", PROCESSED_DIR / "librispeech_test_clean")
m_fl = build_manifest(fl, "transcription", "ru", "fl_ru", PROCESSED_DIR / "fleurs_ru_test")
```

```
m_ls.to_csv(MANIFEST_DIR / "asr_librispeech_test_clean.csv", index=False)
m_fl.to_csv(MANIFEST_DIR / "asr_fleurs_ru_test.csv", index=False)
```

```
ls: 100% 2620/2620 [00:37<00:00, 70.04it/s]
```

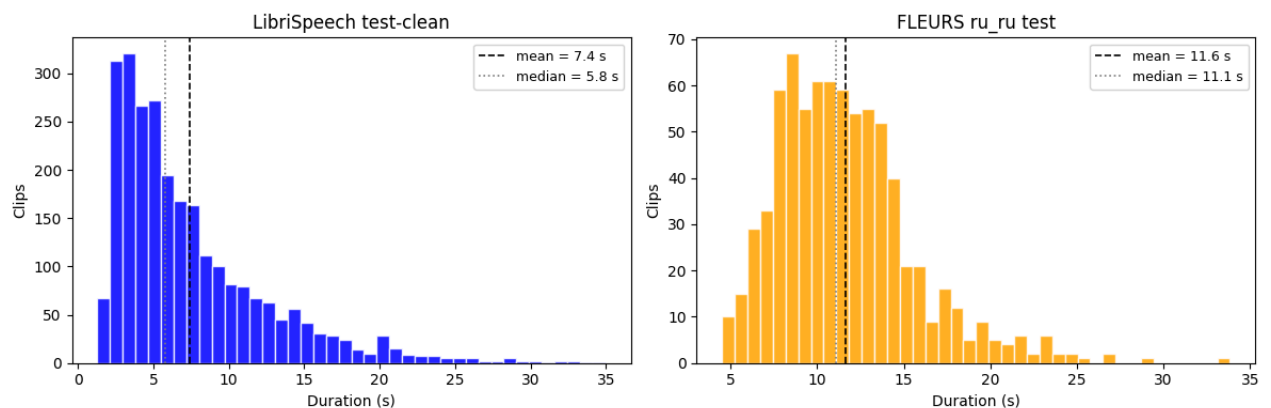
```
fl_ru: 100% 775/775 [00:11<00:00, 65.41it/s]
```

4. EDA — Duration

```
datasets_plot = [
    (m_ls, "LibriSpeech test-clean", "blue"),
    (m_fl, "FLEURS ru_ru test", "orange"),
]

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, (df, title, color) in zip(axes, datasets_plot):
    d = df["duration_sec"]
    ax.hist(d, bins=40, color=color, edgecolor="white", alpha=0.85)
    ax.axvline(d.mean(), color="black", linestyle="--", lw=1.2, label=f"mean = {d.mean():.1f} s")
    ax.axvline(d.median(), color="grey", linestyle=":", lw=1.2, label=f"median = {d.median():.1f} s")
    ax.set_title(title)
    ax.set_xlabel("Duration (s)")
    ax.set_ylabel("Clips")
    ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig(EDA_DIR / "01_duration.png", dpi=150, bbox_inches="tight")
plt.show()
```



```
for df, name in [(m_ls, "LibriSpeech"), (m_fl, "FLEURS-ru")]:
    short = (df["duration_sec"] < 1.0).sum()
    long = (df["duration_sec"] > 30.0).sum()
    total = len(df)
    print(f"{name}:")
    print(f"total clips: {total:,}")
    print(f"total hours: {df['duration_sec'].sum() / 3600:.2f} h")
    print(f"< 1 s: {short} ({100*short/total:.1f}%)")
    print(f"> 30 s: {long} ({100*long/total:.1f}%)")
```

```
LibriSpeech:
total clips: 2,620
total hours: 5.40 h
< 1 s: 0 (0.0%)
> 30 s: 9 (0.3%)
FLEURS-ru:
total clips: 775
total hours: 2.50 h
< 1 s: 0 (0.0%)
> 30 s: 1 (0.1%)
```

5. EDA — Transcript

```
for df, name in [(m_ls, "LibriSpeech"), (m_fl, "FLEURS-ru")]:
    df["words_per_sec"] = df["n_words"] / df["duration_sec"]

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
```

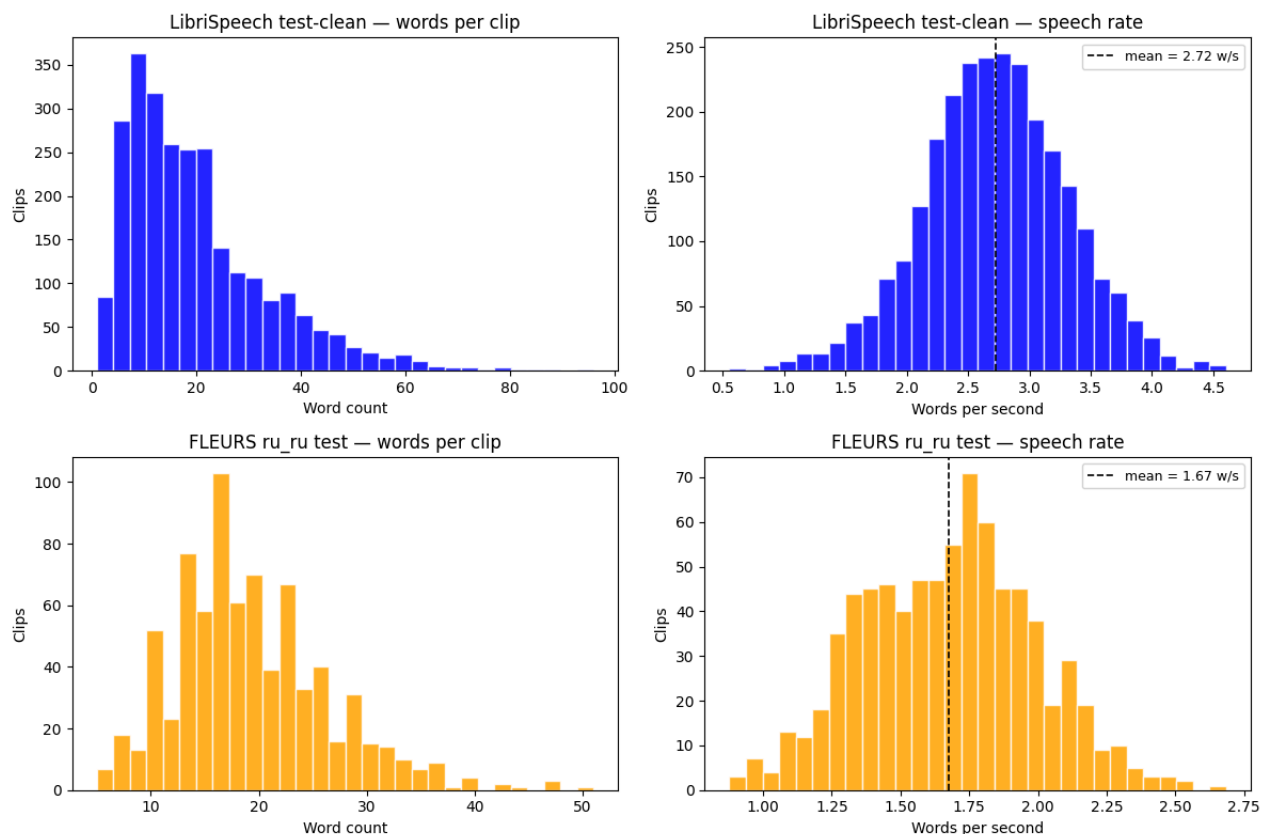
```

for row, (df, name, color) in enumerate(datasets_plot):
    axes[row][0].hist(df["n_words"], bins=30, color=color, edgecolor="white", alpha=0.85)
    axes[row][0].set_title(f"{name} — words per clip")
    axes[row][0].set_xlabel("Word count")
    axes[row][0].set_ylabel("Clips")

    wps = df["words_per_sec"]
    axes[row][1].hist(wps, bins=30, color=color, edgecolor="white", alpha=0.85)
    axes[row][1].axvline(wps.mean(), color="black", linestyle="--", lw=1.2,
                        label=f"mean = {wps.mean():.2f} w/s")
    axes[row][1].set_title(f"{name} — speech rate")
    axes[row][1].set_xlabel("Words per second")
    axes[row][1].set_ylabel("Clips")
    axes[row][1].legend(fontsize=9)

plt.tight_layout()
plt.savefig(EDA_DIR / "02_transcript.png", dpi=150, bbox_inches="tight")
plt.show()

```



6. EDA — Character Set

Important for Russian: Parakeet-TDT may normalize ё to e in its output. If so, the same normalization must be applied to FLEURS references before computing WER/CER. Otherwise CER will be artificially inflated.

```

def char_category_report(df: pd.DataFrame, name: str):
    text = " ".join(df["transcript"].dropna())
    freq = Counter(text)
    cats = {"letter": 0, "digit": 0, "punctuation": 0, "space": 0, "other": 0}
    unusual = Counter()
    for ch, cnt in freq.items():
        cat = unicodedata.category(ch)
        if cat.startswith("L"):
            cats["letter"] += cnt
        elif cat.startswith("N"):
            cats["digit"] += cnt
        elif cat.startswith("P"):

```

```

        cats["punctuation"] += cnt
    elif cat.startswith("Z") or ch in " \t\n":
        cats["space"] += cnt
    else:
        cats["other"] += cnt
        unusual[ch] += cnt

total = sum(cats.values())
print(f"\n{name}:")
for cat, cnt in cats.items():
    print(f"{cat}: {cnt}  ({100*cnt/total:.1f}%)")
if unusual:
    print(f"unusual: {dict(unusual.most_common(10))}")

char_category_report(m_ls, "LibriSpeech EN")
char_category_report(m_fl, "FLEURS RU")

```

```

LibriSpeech EN:
letter: 231030  (81.3%)
digit: 0  (0.0%)
punctuation: 544  (0.2%)
space: 52575  (18.5%)
other: 0  (0.0%)

```

```

FLEURS RU:
letter: 90930  (85.1%)
digit: 664  (0.6%)
punctuation: 241  (0.2%)
space: 14968  (14.0%)
other: 1  (0.0%)
unusual: {'№': 1}

```

```

ru_text = " ".join(m_fl["transcript"].dropna())
yo_count = ru_text.count("ё") + ru_text.count("Ё")
total = len(ru_text)

```

```

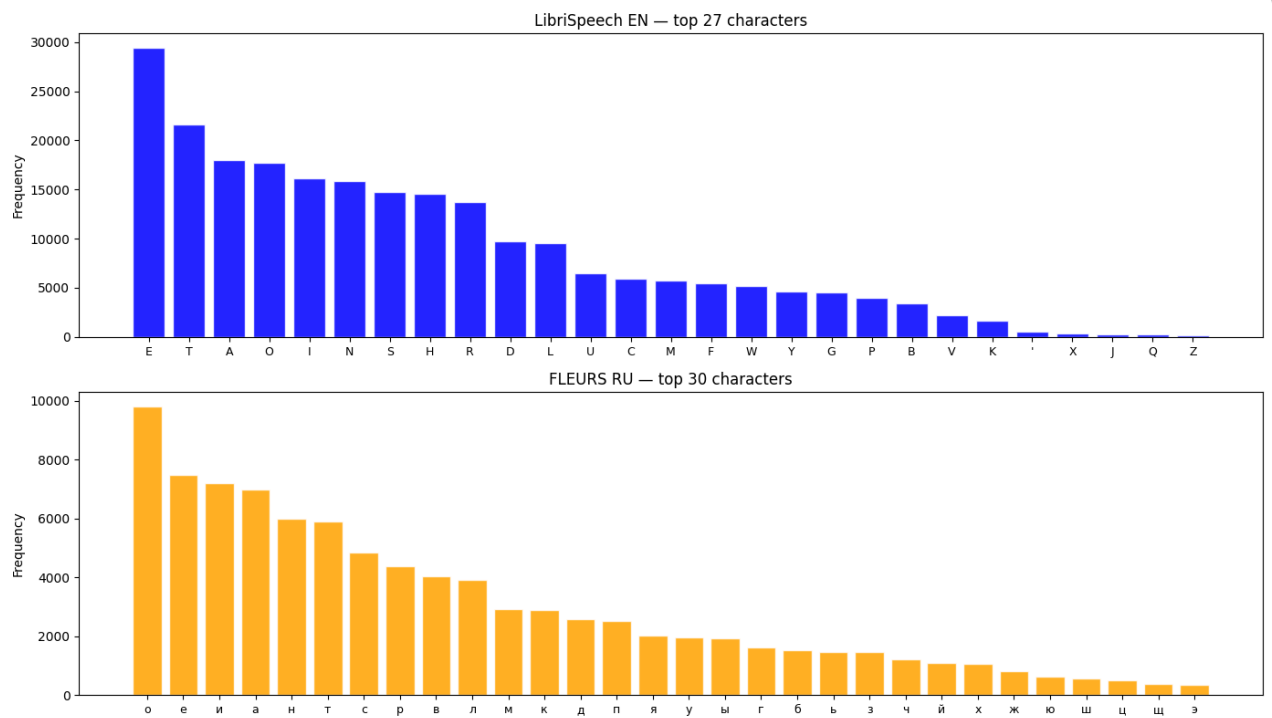
def plot_char_freq(df: pd.DataFrame, name: str, color: str, ax, top_n: int = 30):
    text = " ".join(df["transcript"].dropna())
    freq = Counter(c for c in text if not c.isspace())
    most_common = freq.most_common(top_n)
    n = len(most_common)
    chars, counts = zip(*most_common)
    ax.bar(range(n), counts, color=color, edgecolor="white", alpha=0.85)
    ax.set_xticks(range(n))
    ax.set_xticklabels(chars, fontsize=9)
    ax.set_title(f"{name} – top {n} characters")
    ax.set_ylabel("Frequency")

```

```

fig, axes = plt.subplots(2, 1, figsize=(14, 8))
plot_char_freq(m_ls, "LibriSpeech EN", "blue", axes[0])
plot_char_freq(m_fl, "FLEURS RU", "orange", axes[1])
plt.tight_layout()
plt.savefig(EDA_DIR / "03_character_freq.png", dpi=150, bbox_inches="tight")
plt.show()

```



7. EDA — Amplitude

Near-silent clips (RMS < 0.01) risk producing empty or hallucinated ASR output. We check the raw signal **before preprocessing** to catch source-level issues.

```
def collect_amplitude_stats(dataset, transcript_key: str, desc: str) -> pd.DataFrame:
    rows = []
    for sample in tqdm(dataset, desc=desc):
        arr = np.array(sample["audio"]["array"], dtype=np.float32)
        rows.append({
            "peak": float(np.max(np.abs(arr))),
            "rms" : float(np.sqrt(np.mean(arr ** 2))),
        })
    return pd.DataFrame(rows)
```

```
amp_ls = collect_amplitude_stats(ls, "text", "LibriSpeech amplitude")
amp_fl = collect_amplitude_stats(fl, "transcription", "FLEURS amplitude")
```

LibriSpeech amplitude: 100% 2620/2620 [00:12<00:00, 260.11it/s]

FLEURS amplitude: 100% 775/775 [00:02<00:00, 378.56it/s]

SILENCE_THRESHOLD = 0.01

```
for amp, name in [(amp_ls, "LibriSpeech"), (amp_fl, "FLEURS-ru")]:
    silent = (amp["rms"] < SILENCE_THRESHOLD).sum()
    print(f"{name}: near-silent clips (RMS < {SILENCE_THRESHOLD}) = {silent}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, (amp, name, color) in zip(axes, [
    (amp_ls, "LibriSpeech EN", "blue"),
    (amp_fl, "FLEURS RU", "orange"),
]):
    ax.hist(amp["rms"], bins=40, color=color, edgecolor="white", alpha=0.85)
```